# FloodDiffusion: Tailored Diffusion Forcing for Streaming Motion Generation

**Paper**: arXiv:2512.03520 | **Project Page**: shandaai.github.io/FloodDiffusion

**Authors**: Yiyi Cai, Yuhan Wu, Kunhang Li, You Zhou, Bo Zheng, Haiyang Liu (Shanda AI Research Tokyo)

---

## What does FloodDiffusion do?

FloodDiffusion generates **realistic 3D human motion sequences from text descriptions** in real-time. Given prompts like *"a person walking forward"* or *"a person jumps and then waves"*, it produces frame-by-frame 3D skeleton animations.

### Key Contributions

1. **Streaming generation** -- Continuous motion output with text condition changes mid-sequence
2. **Latent Diffusion Forcing (LDF)** -- Combines diffusion models with a triangular noise schedule for efficient autoregressive latent generation
3. **Real-time capable** -- ~50 FPS model output on GPU, enabling interactive applications
4. **VAE compression** -- Compresses 263D motion features to 4D latent tokens (66x compression)

### Notebook Outline

1. **Setup** -- Install dependencies & load the pretrained model
2. **Architecture Deep Dive** -- VAE + Latent Diffusion Forcing explained
3. **Inference** -- Generate motions from text prompts
4. **Training Overview** -- How the model is trained (conceptual + code)
5. **Results & Paper Comparison** -- Quantitative evaluation metrics
6. **Demo** -- Interactive 3D viewer and video rendering

# 1. Setup

First, let's install the required dependencies and load the pretrained model from Hugging Face. We use the **Tiny** variant which is lighter and faster, ideal for Colab's T4 GPU.

```
# Install dependencies
!pip install -q torch torchvision torchaudio
!pip install -q transformers huggingface_hub sentencepiece protobuf
!pip install -q imageio imageio-ffmpeg matplotlib numpy plotly ipywidge
!pip install -q lightning
```

```
import os, json, math, time, tempfile, uuid
import numpy as np
import torch
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from mpl_toolkits.mplot3d import Axes3D
from IPython.display import HTML, Video, display
import plotly.graph_objects as go
import imageio


def normalize_joints_output(output):
    """Return a single motion sample as a NumPy array of shape (frame
    if isinstance(output, (list, tuple)):
        if len(output) == 0:
            raise ValueError("Model returned an empty output list.")
        if len(output) > 1:
            raise ValueError(
                f"Expected one motion sample but got {len(output)}. "
                "Pass a single prompt/length when a single output is
            )
        output = output[0]

    joints_np = np.asarray(output)
    if joints_np.ndim != 3 or joints_np.shape[-2:] != (22, 3):
        raise ValueError(
            f"Unexpected joint output shape {joints_np.shape}. "
            "Expected (frames, 22, 3)."
        )
    return joints_np


print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    props = torch.cuda.get_device_properties(0)
    print(f"GPU: {props.name}")
    print(f"GPU Memory: {props.total_memory / 1e9:.1f} GB")
```

```
PyTorch version: 2.10.0+cu128
CUDA available: True
GPU: Tesla T4
GPU Memory: 15.6 GB
```

## Load the Pretrained Model

The model is hosted on Hugging Face as `ShandaAI/FloodDiffusionTiny`. The first run downloads the weights (~1-2 GB for tiny).

We apply CPU-compatibility patches so the model can also run without a GPU (just slower).

```
import re
from huggingface_hub import snapshot_download
from transformers import AutoModel

# CPU compatibility: prevent crash at class-definition time in t5.py
if not torch.cuda.is_available():
    torch.cuda.current_device = lambda: 0

def patch_model_files(model_dir):
    """Patch cached model files for CPU-only or non-flash-attention env

    # 1) t5.py -- torch.cuda.current_device() -> safe fallback
    t5 = os.path.join(model_dir, "ldf_models", "tools", "t5.py")
    if os.path.exists(t5):
        src = open(t5).read()
        if "torch.cuda.current_device()" in src:
            src = src.replace(
                "device=torch.cuda.current_device(),",
                'device=torch.device("cuda") if torch.cuda.is_available
            )
            open(t5, "w").write(src)

    # 2) wan_model.py -- flash_attention -> standard attention (SDPA fa
    wan = os.path.join(model_dir, "ldf_models", "tools", "wan_model.py"
    if os.path.exists(wan):
        src = open(wan).read()
        if "from .attention import flash_attention" in src:
            src = src.replace("from .attention import flash_attention",
            src = src.replace("flash_attention(", "attention(")
            open(wan, "w").write(src)

    # 3) attention.py -- preserve input dtype through SDPA path
    attn = os.path.join(model_dir, "ldf_models", "tools", "attention.py
    if os.path.exists(attn):
        src = open(attn).read()
```

```
        src = open(attn).read()
        if "out_dtype = q.dtype" not in src:
            src = src.replace(
                "        q = q.transpose(1, 2).to(dtype)",
                "        out_dtype = q.dtype\n        q = q.transpose(1
            )
            src = src.replace(
                "        out = out.transpose(1, 2).contiguous()\n
                "        out = out.transpose(1, 2).contiguous()\n
            )
            open(attn, "w").write(src)

    # 4) diffusion_forcing_wan_tiny.py -- AutoModel -> UMT5EncoderModel
    tiny = os.path.join(model_dir, "ldf_models", "diffusion_forcing_wan
    if os.path.exists(tiny):
        src = open(tiny).read()
        if "AutoModel.from_pretrained" in src:
            src = src.replace(
                "from transformers import AutoTokenizer, AutoModel",
                "from transformers import AutoTokenizer, UMT5EncoderMod
            )
            src = re.sub(
                r"self\.model\s*=\s*AutoModel\.from_pretrained\(\s*mode
                "self.model = UMT5EncoderModel.from_pretrained(\n"
                "            model_name,\n"
                "            torch_dtype=dtype\n"
                "        )",
                src,
            )
            open(tiny, "w").write(src)

# Download and patch
print("Downloading model from Hugging Face...")
model_id = "ShandaAI/FloodDiffusionTiny"
local_dir = snapshot_download(model_id)
print(f"Model cached at: {local_dir}")

patch_model_files(local_dir)

print("Loading model...")
model = AutoModel.from_pretrained(local_dir, trust_remote_code=True)
print("Model loaded successfully!")
```

```
Downloading model from Hugging Face...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:
Error while fetching `HF_TOKEN` secret value from your vault: 'Requesti
```

```
You are not authenticated with the Hugging Face Hub in this notebook.
If the error persists, please let us know by opening an issue on GitHub
  warnings.warn(
Download complete: 0.00/0.00 [00:00<?, ?B/s]
Fetching 24 files: 100% 24/24 [00:00<00:00, 690.03it/s]
Model cached at: /root/.cache/huggingface/hub/models--ShandaAI--FloodDi
Loading model...
INFO: Seed set to 1234
INFO:lightning.fabric.utilities.seed:Seed set to 1234
Loaded VAE model from /root/.cache/huggingface/hub/models--ShandaAI--Fl
Only in state_dict (not in parameters): ['mean', 'std']
Total state_dict items: 108
Total named_parameters: 106
Total named_buffers: 2
Warning: You are sending unauthenticated requests to the HF Hub. Please
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenti
Loading google/umt5-base from HuggingFace...
Loading google/umt5-base from HuggingFace...
Loading weights: 100% 123/123 [00:01<00:00, 277.77it/s, Materializing param=shared.weight]
UMT5EncoderModel LOAD REPORT from: google/umt5-base
Key
------------------------------------------------------------------------
decoder.block.{0...11}.layer.0.SelfAttention.k.weight
decoder.block.{0...11}.layer.1.EncDecAttention.k.weight
decoder.block.{0...11}.layer.1.EncDecAttention.q.weight
decoder.block.{0...11}.layer.{0, 1, 2}.layer_norm.weight
decoder.block.{0...11}.layer.2.DenseReluDense.wi_0.weight
decoder.block.{0...11}.layer.1.EncDecAttention.o.weight
decoder.block.{0...11}.layer.0.SelfAttention.o.weight
decoder.block.{0...11}.layer.0.SelfAttention.relative_attention_bias.we
decoder.block.{0...11}.layer.0.SelfAttention.q.weight
decoder.block.{0...11}.layer.0.SelfAttention.v.weight
decoder.block.{0...11}.layer.2.DenseReluDense.wi_1.weight
decoder.block.{0...11}.layer.1.EncDecAttention.v.weight
decoder.embed_tokens.weight
decoder.block.{0...11}.layer.2.DenseReluDense.wo.weight
lm_head.weight
decoder.final_layer_norm.weight

Notes:
- UNEXPECTED    :can be ignored when loading from different task/archit
Loaded model from /root/.cache/huggingface/hub/models--ShandaAI--FloodD
All parameters match between state_dict and named_parameters
Total state_dict items: 231
Total named_parameters: 231
Total named_buffers: 0
Model loaded successfully!
```

## 2. Architecture Deep Dive

FloodDiffusion has a **two-stage architecture**:

### Stage 1: Motion VAE (Variational Autoencoder)

Compresses high-dimensional motion data into a compact latent space.

```
Input: T x 263 motion features ——> Encoder ——> (T/4) x 4 latent tokens —
```

**The 263-dimensional motion representation** (HumanML3D format):

| Component | Dimensions | Description |
|---|---|---|
| Root rotation velocity | 1 | Y-axis angular velocity |
| Root linear velocity | 2 | XZ plane velocity |
| Root Y position | 1 | Height of root joint |
| Joint positions (RIC) | 63 | Relative positions of 21 joints (21 x 3) |
| Joint rotations | 126 | 6D rotation of 21 joints (21 x 6) |
| Local joint velocity | 66 | Velocity of 22 joints (22 x 3) |
| Foot contact | 4 | Binary contact labels for 4 foot joints |

The VAE uses a **causal 1D convolutional architecture** (based on WAN2.2) with:

- **2-stage temporal downsampling** (4x compression: T -> T/4)
- **Latent dimension**: 4 (from 263 input)
- **Loss**: Reconstruction (SmoothL1) + Velocity + KL divergence (beta-VAE)

### Stage 2: Latent Diffusion Forcing (LDF)

Generates latent motion tokens autoregressively using a diffusion process.

```
Text prompt ——> T5 Encoder ——> Text features (4096D)
                                        |
                                        v
Noise ——> Triangular Noise Schedule ——> DiT Transformer ——> Denoised late
                                                                        |
```

```
                                                                    v
                                                            VAE Decoder -
```

**Key innovation -- Triangular Noise Schedule**: Unlike standard diffusion that denoises the entire sequence at once, LDF uses a **sliding triangular window**:

- Tokens on the **left** are clean (already generated)
- Tokens on the **right** are noisy (pure noise)
- A **gradient transition** in between

This enables **streaming**: as the window slides right, new motion is progressively generated while earlier frames are committed.

## Model Components (Tiny variant)

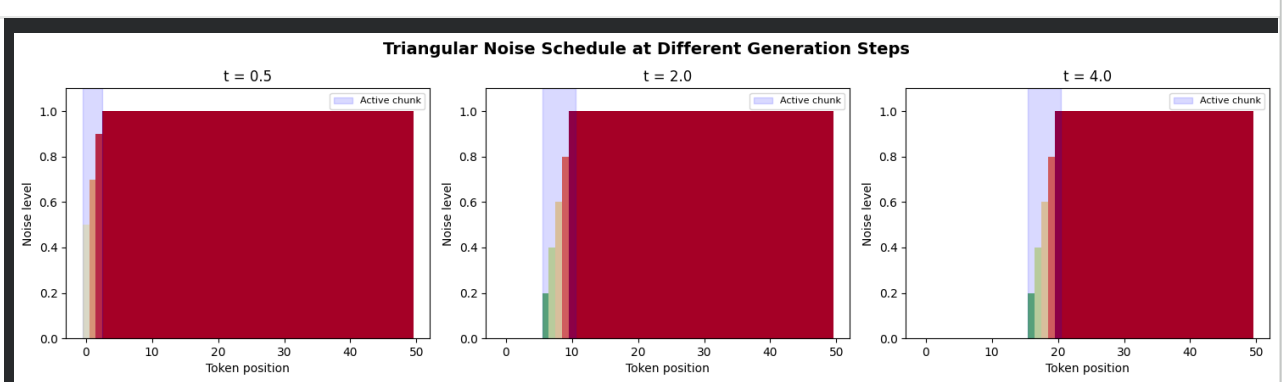
| Component | Details |
|---|---|
| Text Encoder | UMT5-base (512D output) |
| Backbone | WAN DiT Transformer |
| Hidden dim | 512 |
| Attention heads | 8 |
| Transformer layers | 4 |
| Chunk size | 5 tokens |
| Noise steps | 10 |
| Prediction type | Velocity (v-prediction) |
| CFG scale | 5.0 |

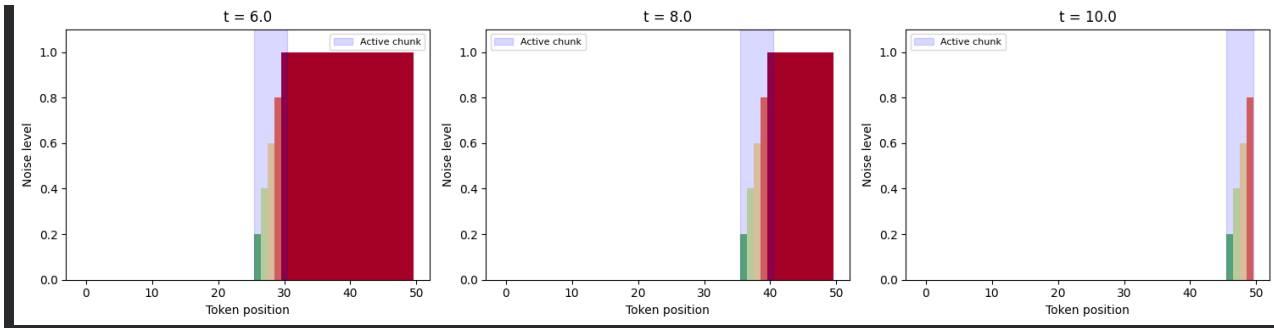
## Visualizing the Triangular Noise Schedule

The key insight of Diffusion Forcing is the **triangular noise schedule**. Let's visualize how noise levels vary across the sequence at different time steps.

```
def get_noise_levels(seq_len, time_step, chunk_size=5):
    """Replicate the triangular noise schedule from DiffForcingWanModel
    noise_level = np.clip(
        1 + np.arange(seq_len) / chunk_size - time_step,
        0.0, 1.0
    )
    return noise_level
```

```
# Visualize noise levels at different generation time steps
seq_len = 50
chunk_size = 5
num_denoise_steps = 10
dt = 1.0 / num_denoise_steps

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
fig.suptitle("Triangular Noise Schedule at Different Generation Steps",

time_steps_to_show = [0.5, 2.0, 4.0, 6.0, 8.0, 10.0]
for ax, t in zip(axes.flat, time_steps_to_show):
    noise = get_noise_levels(seq_len, t, chunk_size)
    colors = plt.cm.RdYlGn_r(noise)  # Red = noisy, Green = clean

    ax.bar(range(seq_len), noise, color=colors, width=1.0, edgecolor='n
    ax.set_ylim(0, 1.1)
    ax.set_xlabel("Token position")
    ax.set_ylabel("Noise level")
    ax.set_title(f"t = {t:.1f}")

    # Mark the "active chunk" being denoised
    start_idx = max(0, int(chunk_size * (t - 1)) + 1)
    end_idx = int(chunk_size * t) + 1
    ax.axvspan(start_idx - 0.5, min(end_idx, seq_len) - 0.5, alpha=0.15
    ax.legend(fontsize=8)

plt.tight_layout()
plt.show()

print("The triangular schedule creates a sliding window:")
print("  - LEFT of window: noise_level = 0 (clean, already generated)")
print("  - INSIDE window: noise_level transitions from 0 to 1 (being de
print("  - RIGHT of window: noise_level = 1 (pure noise, not yet genera
print(f"\nWith chunk_size={chunk_size} and noise_steps={num_denoise_ste
print(f"  Each chunk gets {num_denoise_steps} denoising iterations befo
```






```
The triangular schedule creates a sliding window:
  - LEFT of window: noise_level = 0 (clean, already generated)
  - INSIDE window: noise_level transitions from 0 to 1 (being denoised)
  - RIGHT of window: noise_level = 1 (pure noise, not yet generated)

With chunk_size=5 and noise_steps=10:
  Each chunk gets 10 denoising iterations before being committed.
```

## Model Architecture Inspection

Let's look at the actual model components and their parameter counts.

```
# Inspect model structure
print("=" * 60)
print("FloodDiffusion Model Components")
print("=" * 60)

# Handle both naming conventions for the VAE attribute
ldf_model = model.ldf_model  # Latent Diffusion Forcing model
```

```
ldf_model = model.ldf_model  # Latent Diffusion Forcing model
vae_model = getattr(model, "vae", None) or getattr(model, "vae_model",
if vae_model is None:
    raise AttributeError("Could not find VAE module on model (expected

def count_params(m):
    return sum(p.numel() for p in m.parameters())

def maybe_attr(obj, name):
    return getattr(obj, name, "N/A")

print()
print("--- VAE (Motion Encoder/Decoder) ---")
print(f"  Total parameters: {count_params(vae_model):,}")
print(f"  Input dim: {maybe_attr(vae_model, 'input_dim')}")
print(f"  Latent dim: {maybe_attr(vae_model, 'latent_dim')}")
print(f"  Temporal downsampling: 4x")

print()
print("--- LDF (Latent Diffusion Forcing) ---")
print(f"  Total parameters: {count_params(ldf_model):,}")
print(f"  Input dim (latent): {maybe_attr(ldf_model, 'input_dim')}")
print(f"  Hidden dim: {maybe_attr(ldf_model, 'hidden_dim')}")
print(f"  Num heads: {maybe_attr(ldf_model, 'num_heads')}")
print(f"  Num layers: {maybe_attr(ldf_model, 'num_layers')}")
print(f"  Chunk size: {maybe_attr(ldf_model, 'chunk_size')}")
print(f"  Noise steps: {maybe_attr(ldf_model, 'noise_steps')}")
print(f"  Prediction type: {maybe_attr(ldf_model, 'prediction_type')}")
print(f"  CFG scale: {maybe_attr(ldf_model, 'cfg_scale')}")

text_encoder = maybe_attr(ldf_model, "text_encoder")
if text_encoder != "N/A":
    print()
    print("--- Text Encoder ---")
    text_encoder_model = getattr(text_encoder, "model", text_encoder)
    print(f"  Total parameters: {count_params(text_encoder_model):,}")

total = count_params(vae_model) + count_params(ldf_model)
print()
print("--- Total Model ---")
print(f"  Total parameters: {total:,}")
print(f"  Total size (approx): {total * 4 / 1e9:.2f} GB (fp32)")
```

```
============================================================
```

```
FloodDiffusion Model Components
============================================================

--- VAE (Motion Encoder/Decoder) ---
  Total parameters: 17,503,179
  Input dim: 263
  Latent dim: N/A
  Temporal downsampling: 4x

--- LDF (Latent Diffusion Forcing) ---
  Total parameters: 9,182,468
  Input dim (latent): 4
  Hidden dim: 256
  Num heads: 8
  Num layers: 8
  Chunk size: 5
  Noise steps: 10
  Prediction type: vel
  CFG scale: 5.0

--- Text Encoder ---
  Total parameters: 478,764,288

--- Total Model ---
  Total parameters: 26,685,647
  Total size (approx): 0.11 GB (fp32)
```

---

# 3. Inference -- Generating Motion from Text

Now let's generate some motions! The model takes a text prompt and a length parameter (in latent tokens, where 1 token = 4 frames at 20 FPS).

## 3.1 Single Text Prompt

```
# Generate motion from a single text prompt
prompt = "a person walking forward"
length = 30  # latent tokens (~120 frames at 20 FPS = ~6 seconds)

print(f"Prompt: '{prompt}'")
print(f"Requested length: {length} latent tokens")
print(f"Expected frames: ~{length * 4} (at 4x upsampling)")
print(f"Expected duration: ~{length * 4 / 20:.1f}s (at 20 FPS)")
print()

t0 = time.time()
# output_joints=True returns (N, 22, 3) joint coordinates instead of
joints = model(prompt, length=length, output_joints=True, smoothing_a
elapsed = time.time() - t0

joints = normalize_joints_output(joints)
print(f"Generated joints shape: {joints.shape}")
print(f"  -> {joints.shape[0]} frames, {joints.shape[1]} joints, {joi
print(f"Generation time: {elapsed:.2f}s")
print(f"Generation speed: {joints.shape[0] / elapsed:.1f} frames/s")
```

```
Prompt: 'a person walking forward'
Requested length: 30 latent tokens
Expected frames: ~120 (at 4x upsampling)
Expected duration: ~6.0s (at 20 FPS)

/root/.cache/huggingface/hub/models--ShandaAI--FloodDiffusionTiny/snap
  warnings.warn(
Generated joints shape: (117, 22, 3)
  -> 117 frames, 22 joints, 3D coordinates
Generation time: 5.26s
Generation speed: 22.3 frames/s
```

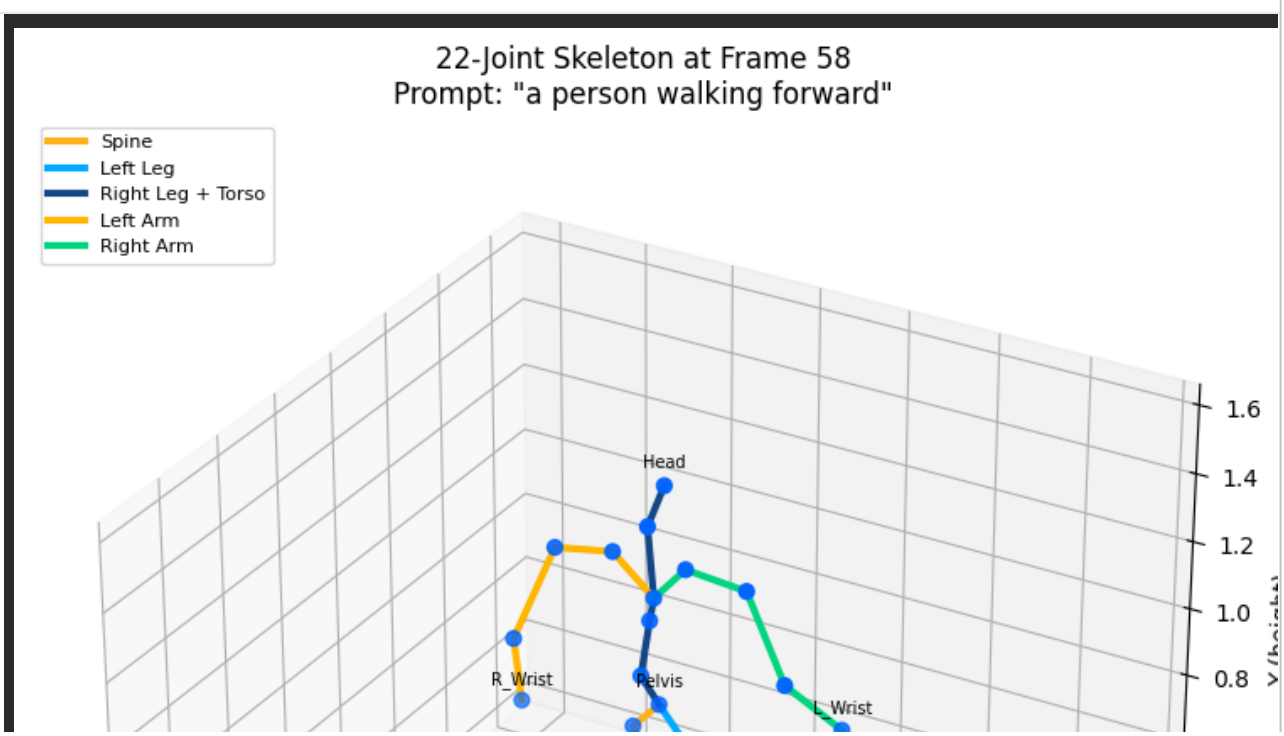
## 3.2 Understanding the Output: 22-Joint Skeleton

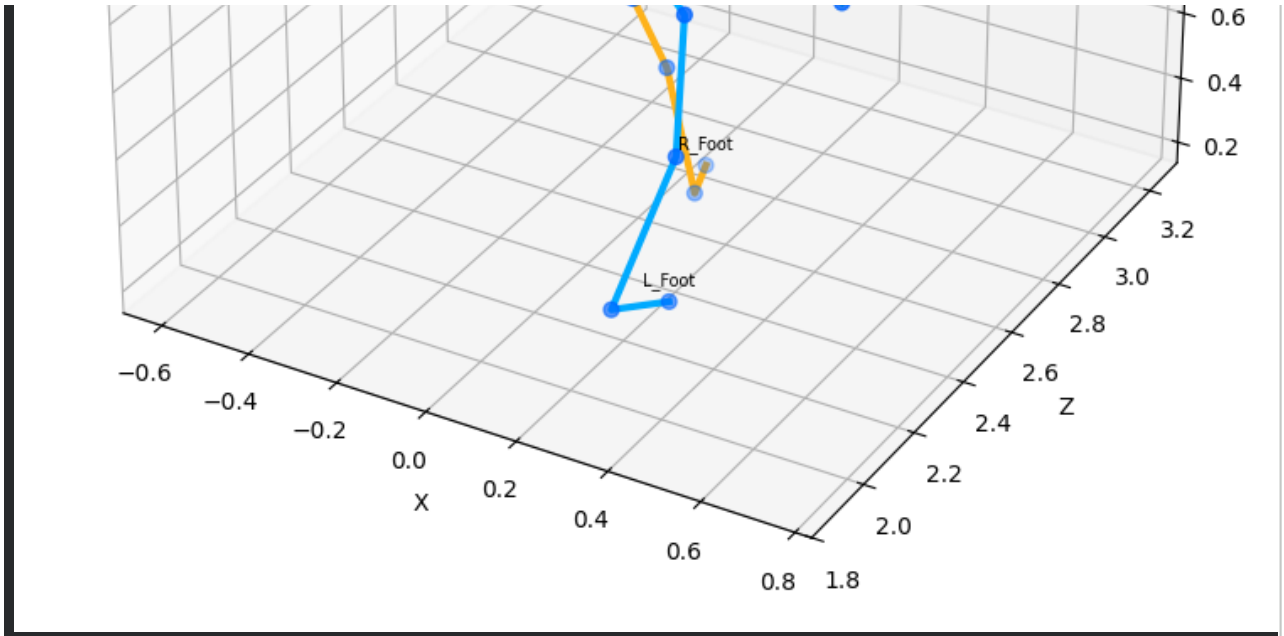
The HumanML3D skeleton has 22 joints connected in 5 kinematic chains:

```
# HumanML3D skeleton definition
JOINT_NAMES = [
    "Pelvis",       # 0
    "L_Hip",        # 1
    "R_Hip",        # 2
```

```
    "Spine1",       # 3  (note: actual chain is 0->2->5->8->11 for spin
    "L_Knee",       # 4
    "R_Knee",       # 5
    "Spine2",       # 6
    "L_Ankle",      # 7
    "R_Ankle",      # 8
    "Spine3",       # 9
    "L_Foot",       # 10
    "R_Foot",       # 11
    "Neck",         # 12
    "L_Collar",     # 13
    "R_Collar",     # 14
    "Head",         # 15
    "L_Shoulder",   # 16
    "R_Shoulder",   # 17
    "L_Elbow",      # 18
    "R_Elbow",      # 19
    "L_Wrist",      # 20
    "R_Wrist",      # 21
]

# Kinematic chains (bone connections)
CHAINS = [
    [0, 2, 5, 8, 11],       # Spine chain (pelvis -> spine -> head area
    [0, 1, 4, 7, 10],       # Left leg (pelvis -> hip -> knee -> ankle
    [0, 3, 6, 9, 12, 15],   # Right leg + torso -> head
    [9, 14, 17, 19, 21],    # Left arm (spine3 -> collar -> shoulder ->
    [9, 13, 16, 18, 20],    # Right arm
]

CHAIN_NAMES = ["Spine", "Left Leg", "Right Leg + Torso", "Left Arm", "R
CHAIN_COLORS = ['#FEB21A', '#00AAFF', '#134686', '#FFB600', '#00D47E']

# Plot the skeleton at a single frame
frame_idx = joints.shape[0] // 2  # middle frame
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')

frame = joints[frame_idx]

# Plot bones
for ci, (chain, color, name) in enumerate(zip(CHAINS, CHAIN_COLORS, CHA
    xs = [frame[j][0] for j in chain]
    ys = [frame[j][1] for j in chain]
    zs = [frame[j][2] for j in chain]
```

```
        ax.plot(xs, zs, ys, color=color, linewidth=3, label=name)

    # Plot joints
    ax.scatter(frame[:, 0], frame[:, 2], frame[:, 1], c='#0064FF', s=40, zo

    # Label a few key joints
    for idx in [0, 15, 20, 21, 10, 11]:
        ax.text(frame[idx, 0], frame[idx, 2], frame[idx, 1] + 0.05,
                JOINT_NAMES[idx], fontsize=7, ha='center')

    ax.set_xlabel('X')
    ax.set_ylabel('Z')
    ax.set_zlabel('Y (height)')
    ax.set_title(f'22-Joint Skeleton at Frame {frame_idx}\nPrompt: "{prompt
    ax.legend(loc='upper left', fontsize=8)

    # Equal aspect ratio
    max_range = np.array([frame[:, 0].max() - frame[:, 0].min(),
                          frame[:, 1].max() - frame[:, 1].min(),
                          frame[:, 2].max() - frame[:, 2].min()]).max() /
    mid = frame.mean(axis=0)
    ax.set_xlim(mid[0] - max_range, mid[0] + max_range)
    ax.set_ylim(mid[2] - max_range, mid[2] + max_range)
    ax.set_zlim(mid[1] - max_range, mid[1] + max_range)

    plt.tight_layout()
    plt.show()
```






## 3.3 Multi-Text Transitions

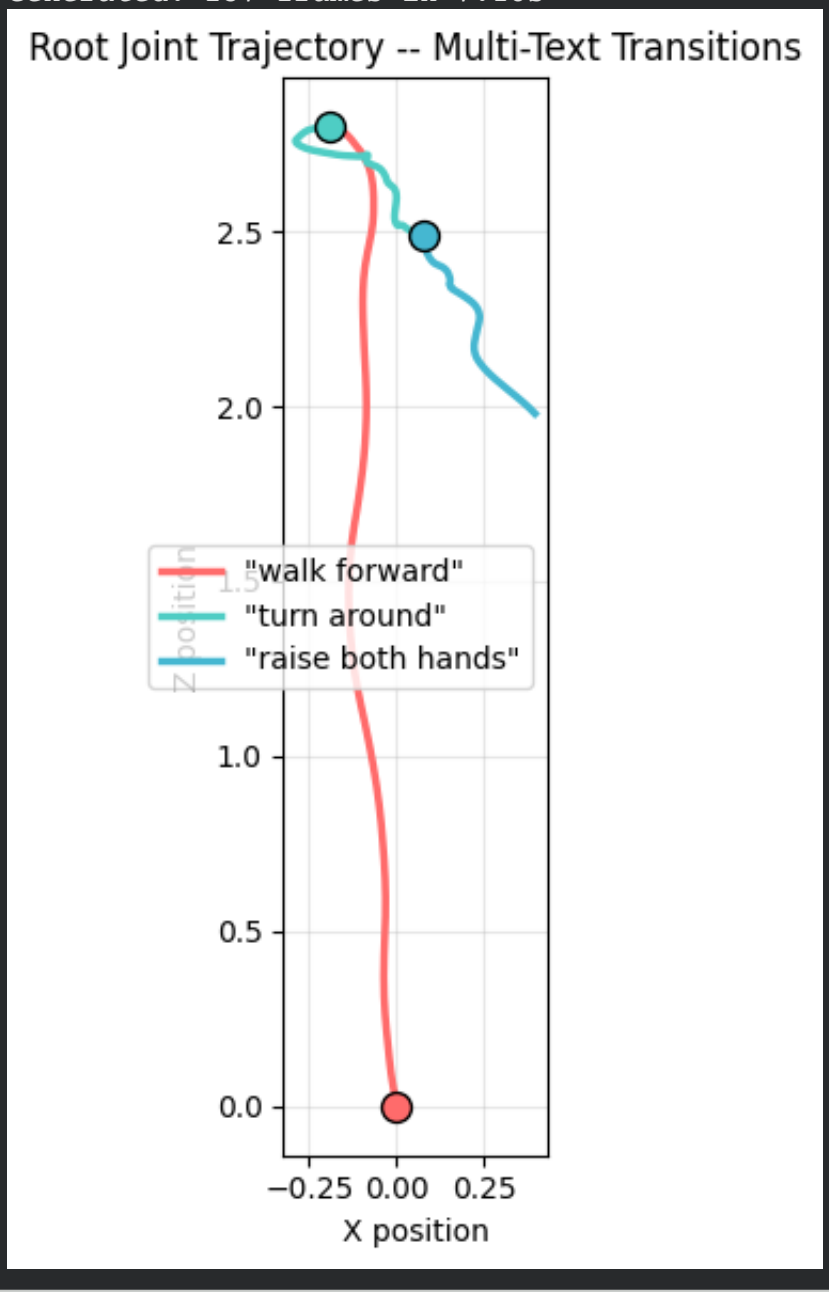
FloodDiffusion supports **seamless transitions** between different text prompts within a single sequence. This is its key advantage over prior work -- the model can change behavior mid-generation.

```
# Multi-text transition generation
text_prompts = ["walk forward", "turn around", "raise both hands"]
text_end = [20, 40, 60]  # latent token boundaries for each prompt
total_length = 60  # total latent tokens

print("Multi-text generation:")
for i, (prompt_i, end) in enumerate(zip(text_prompts, text_end)):
    start = 0 if i == 0 else text_end[i-1]
    frames = (end - start) * 4
    print(f"  [{start*4:3d} - {end*4:3d}] ({frames:3d} frames, {frame
```

```
t0 = time.time()
# Pass a single sample so output is one sequence (not a batch list)
multi_joints = model(
    text=text_prompts,
    length=total_length,
    text_end=text_end,
    output_joints=True,
    smoothing_alpha=0.5,
)
elapsed = time.time() - t0

multi_joints = normalize_joints_output(multi_joints)
print()
print(f"Generated: {multi_joints.shape[0]} frames in {elapsed:.2f}s")

# Plot trajectory (root joint XZ path) to show transitions
root_traj = multi_joints[:, 0, [0, 2]]  # root joint, XZ plane

fig, ax = plt.subplots(figsize=(10, 6))
colors_traj = ['#FF6B6B', '#4ECDC4', '#45B7D1']

for i, (prompt_i, end) in enumerate(zip(text_prompts, text_end)):
    start = 0 if i == 0 else text_end[i-1]
    s_frame, e_frame = start * 4, min(end * 4, len(root_traj))
    if s_frame < len(root_traj):
        ax.plot(root_traj[s_frame:e_frame, 0], root_traj[s_frame:e_fra
                color=colors_traj[i], linewidth=2.5, label=f'"{prompt_
        # Mark start point
        ax.scatter(root_traj[s_frame, 0], root_traj[s_frame, 1],
                   color=colors_traj[i], s=100, zorder=5, edgecolors=

ax.set_xlabel('X position')
ax.set_ylabel('Z position')
ax.set_title('Root Joint Trajectory -- Multi-Text Transitions')
ax.legend()
ax.set_aspect('equal')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Multi-text generation:
  [  0 -  80] ( 80 frames, 4.0s): 'walk forward'
  [ 80 - 160] ( 80 frames, 4.0s): 'turn around'
  [160 - 240] ( 80 frames, 4.0s): 'raise both hands'
```

```
Generated: 237 frames in 7.18s
```




## 3.4 Comparing Different Prompts

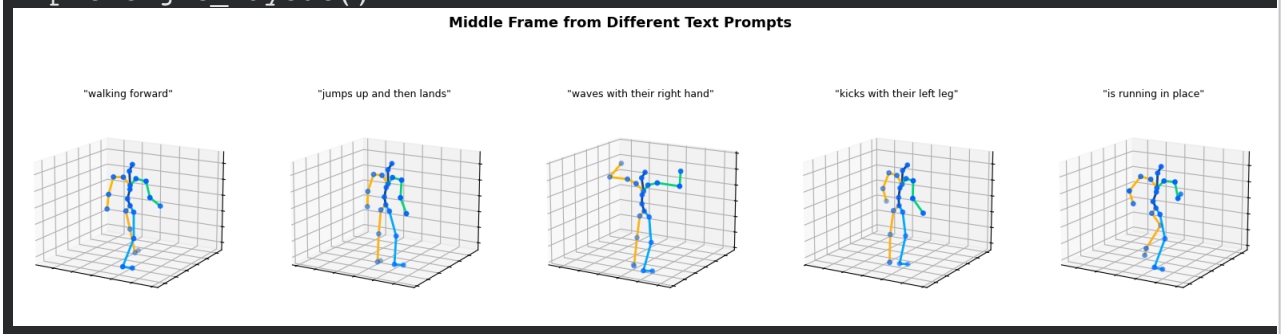
Let's generate several different motions and compare their characteristics.

```
# Generate multiple different motions
test_prompts = [
    "a person walking forward",
    "a person jumps up and then lands",
    "a person waves with their right hand",
```

```
        "a person kicks with their left leg",
        "a person is running in place",
    ]

    results = {}
    for p in test_prompts:
        t0 = time.time()
        j = model(p, length=30, output_joints=True, smoothing_alpha=0.5)
        t1 = time.time()
        results[p] = {"joints": normalize_joints_output(j), "time": t1 - 
        print(f"  '{p}' -> {results[p]['joints'].shape[0]} frames in {resu

    # Visualize key frames from each motion side by side
    fig, axes = plt.subplots(1, len(test_prompts), figsize=(20, 5), subplo

    for ax, p in zip(axes, test_prompts):
        j = results[p]["joints"]
        frame = j[j.shape[0] // 2]  # middle frame

        for ci, (chain, color) in enumerate(zip(CHAINS, CHAIN_COLORS)):
            xs = [frame[jj][0] for jj in chain]
            ys = [frame[jj][1] for jj in chain]
            zs = [frame[jj][2] for jj in chain]
            ax.plot(xs, zs, ys, color=color, linewidth=2)

        ax.scatter(frame[:, 0], frame[:, 2], frame[:, 1], c='#0064FF', s=

        # Set consistent viewing angle
        ax.view_init(elev=15, azim=-60)
        short_title = p.replace("a person ", "").replace("a person is ", "
        ax.set_title(f'"{short_title}"', fontsize=9)
        ax.set_xticklabels([])
        ax.set_yticklabels([])
        ax.set_zticklabels([])

        max_range = np.array([frame[:, 0].max() - frame[:, 0].min(),
                              frame[:, 1].max() - frame[:, 1].min(),
                              frame[:, 2].max() - frame[:, 2].min()]).max
        mid = frame.mean(axis=0)
        ax.set_xlim(mid[0] - max_range, mid[0] + max_range)
        ax.set_ylim(mid[2] - max_range, mid[2] + max_range)
        ax.set_zlim(mid[1] - max_range, mid[1] + max_range)

    plt.suptitle("Middle Frame from Different Text Prompts", fontsize=13,
    plt.tight_layout()
```

```
plt.show()
```

```
  'a person walking forward' -> 117 frames in 4.40s
  'a person jumps up and then lands' -> 117 frames in 3.72s
  'a person waves with their right hand' -> 117 frames in 3.72s
  'a person kicks with their left leg' -> 117 frames in 4.30s
  'a person is running in place' -> 117 frames in 3.66s
/tmp/ipykernel_7354/311682213.py:50: UserWarning: Tight layout not appl
  plt.tight_layout()
```




---

## 4. Training Overview

FloodDiffusion is trained in **three phases**. Here we walk through each phase conceptually and show the key training code.

### Phase 1: Train the VAE (Motion Encoder/Decoder)

The VAE learns to compress 263D motion features into 4D latent tokens.

**Training details:**

- Dataset: HumanML3D (~15,000 motion sequences)
- Batch size: 256
- Training steps: 3,000,000

- Optimizer: AdamW (lr=2e-4)
- Loss: Reconstruction (SmoothL1) + Velocity matching + KL divergence

```
# VAE Loss (from models/vae_wan_1d.py):
total_loss = (
    LAMBDA_FEATURE * loss_recons      # Reconstruction: SmoothL1(decoded,
    + LAMBDA_VELOCITY * loss_vel      # Velocity: SmoothL1 on joint veloc
    + LAMBDA_KL * kl_loss             # KL divergence: regularize latent
)
# where LAMBDA_FEATURE=1.0, LAMBDA_VELOCITY=0.5, LAMBDA_KL=10e-6
```

## Phase 2: Pre-tokenize the Dataset

Once the VAE is trained, we encode the entire dataset into 4D latent tokens. This avoids running the VAE encoder during diffusion training.

```
python pretokenize_vae.py --config configs/vae_wan_1d.yaml
# Creates: raw_data/HumanML3D/TOKENS_*/  (4D tokens for each motion)
```

## Phase 3: Train the Latent Diffusion Forcing (LDF) Model

The LDF model learns to generate latent tokens from text using diffusion forcing.

**Training details:**

- Input: Pre-tokenized 4D features + text descriptions
- Batch size: 16
- Training steps: 300,000 (HumanML3D) / 240,000 (BABEL streaming)
- Optimizer: AdamW (lr=2e-4)
- Scheduler: Cosine annealing
- Loss: MSE on velocity prediction

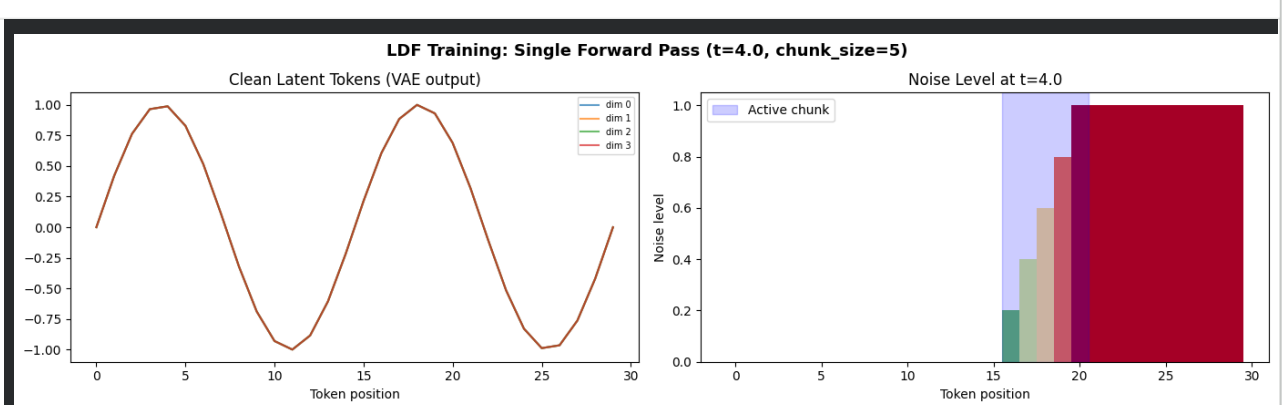
```
# LDF Training loop (from models/diffusion_forcing_wan.py):
# 1. Sample random time step t for each batch element
# 2. Compute triangular noise levels based on t
# 3. Add noise to the latent tokens
# 4. Predict the velocity (clean - noise) through the DiT transformer
# 5. Compute MSE loss on the active chunk (last chunk_size tokens)
```

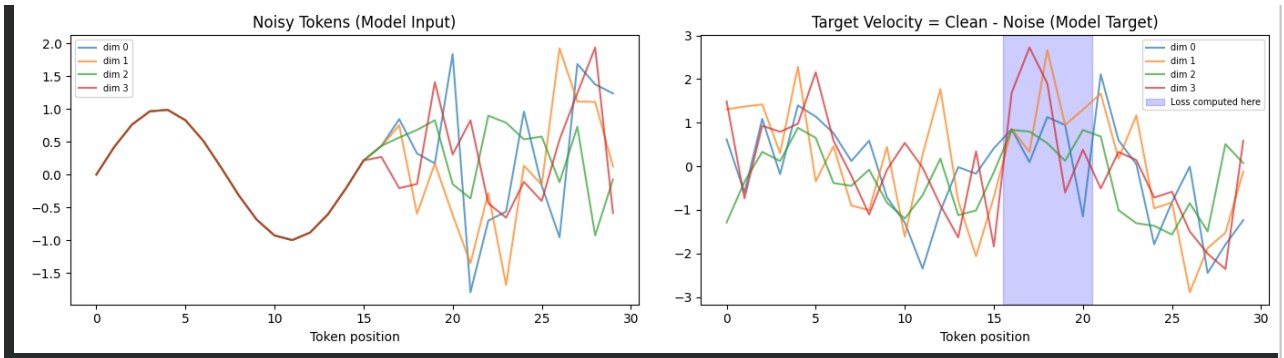
## Training Simulation: Understanding the LDF Forward Pass

Let's simulate a single training step to understand how noise is added and how the model learns.

```
# Simulate the LDF training forward pass (conceptual illustration)
# This shows HOW noise is added during training

chunk_size = 5
seq_len = 30
input_dim = 4

# Simulate a "clean" latent sequence (what the VAE would produce)
clean_tokens = torch.sin(torch.linspace(0, 4 * np.pi, seq_len)).unsque

# Sample a random time step (as done in training)
t = 4.0  # time step
noise_level = np.clip(1 + np.arange(seq_len) / chunk_size - t, 0.0, 1
noise_level_t = torch.tensor(noise_level).float().unsqueeze(1)  # (T,

# Add noise according to the triangular schedule
noise = torch.randn_like(clean_tokens)
noisy_tokens = clean_tokens * (1 - noise_level_t) + noise_level_t * no

# The model predicts velocity = clean - noise
target_velocity = clean_tokens - noise

# Visualize
fig, axes = plt.subplots(2, 2, figsize=(14, 8))

# 1. Clean tokens
ax = axes[0, 0]
for d in range(input_dim):
    ax.plot(clean_tokens[:, d].numpy(), alpha=0.7, label=f'dim {d}')
ax.set_title("Clean Latent Tokens (VAE output)")
ax.set_xlabel("Token position")
ax.legend(fontsize=7)

# 2. Noise levels
ax = axes[0, 1]
```

```
ax.bar(range(seq_len), noise_level, color=plt.cm.RdYlGn_r(noise_level
ax.set_title(f"Noise Level at t={t}")
ax.set_xlabel("Token position")
ax.set_ylabel("Noise level")
active_start = max(0, int(chunk_size * (t - 1)) + 1)
active_end = int(chunk_size * t) + 1
ax.axvspan(active_start - 0.5, active_end - 0.5, alpha=0.2, color='blu
ax.legend()

# 3. Noisy tokens (model input)
ax = axes[1, 0]
for d in range(input_dim):
    ax.plot(noisy_tokens[:, d].numpy(), alpha=0.7, label=f'dim {d}')
ax.set_title("Noisy Tokens (Model Input)")
ax.set_xlabel("Token position")
ax.legend(fontsize=7)

# 4. Target velocity (what the model learns to predict)
ax = axes[1, 1]
for d in range(input_dim):
    ax.plot(target_velocity[:, d].numpy(), alpha=0.7, label=f'dim {d}
ax.axvspan(active_start - 0.5, active_end - 0.5, alpha=0.2, color='blu
ax.set_title("Target Velocity = Clean - Noise (Model Target)")
ax.set_xlabel("Token position")
ax.legend(fontsize=7)

plt.suptitle(f"LDF Training: Single Forward Pass (t={t}, chunk_size={c
plt.tight_layout()
plt.show()

print("Training objective:")
print(f"  1. Sample time t={t} -> noise levels go from 0 (left) to 1
print(f"  2. Add noise: noisy = clean * (1 - noise_level) + noise * no
print(f"  3. Model predicts velocity for the ACTIVE CHUNK (tokens {act
print(f"  4. Loss = MSE(predicted_velocity, target_velocity) on the ac
```





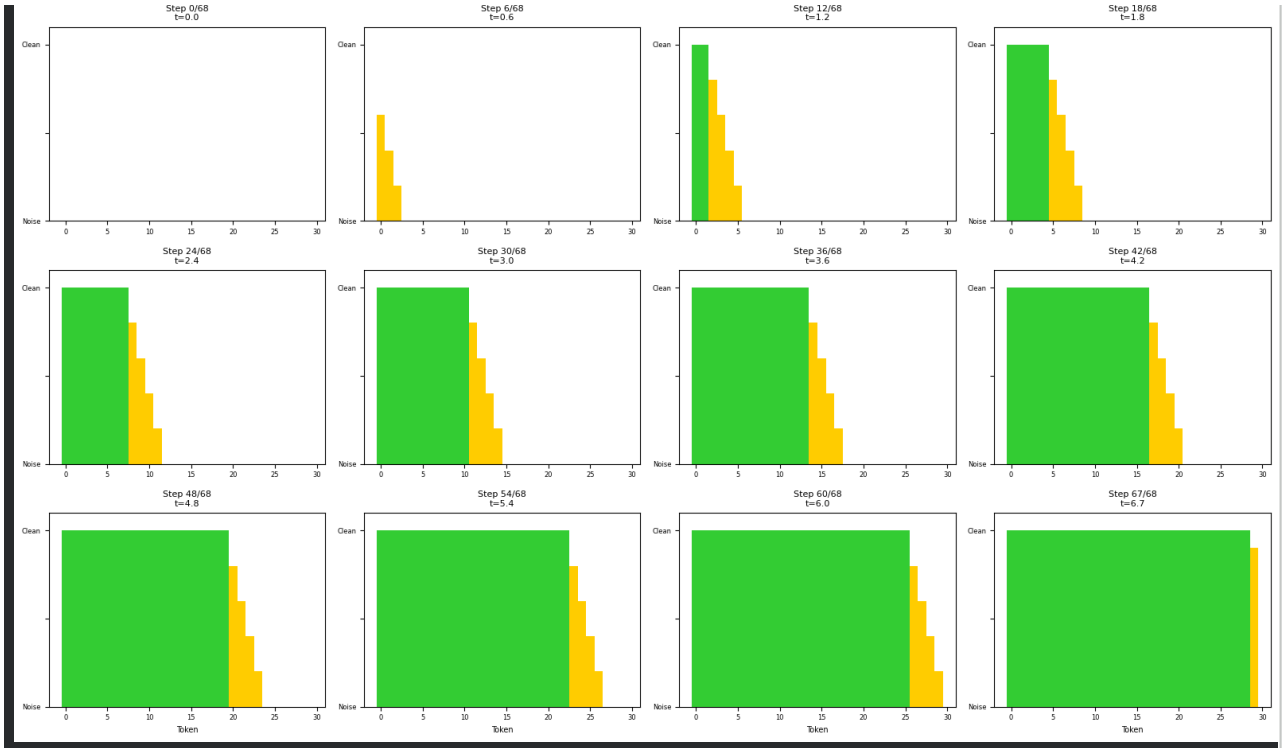

```
Training objective:
  1. Sample time t=4.0 -> noise levels go from 0 (left) to 1 (right)
  2. Add noise: noisy = clean * (1 - noise_level) + noise * noise_level
  3. Model predicts velocity for the ACTIVE CHUNK (tokens 16-21)
  4. Loss = MSE(predicted_velocity, target_velocity) on the active chun
```

## Generation Process Simulation

During inference, the model iteratively denoises the sequence from left to right. Let's visualize how the generation process works step-by-step.

```
# Simulate the generation (inference) process
seq_len = 30
chunk_size = 5
noise_steps = 10
dt = 1.0 / noise_steps
```

```
max_t = 1 + (seq_len - 1) / chunk_size
total_steps = int(max_t / dt)

# Track what happens at each step
fig, axes = plt.subplots(3, 4, figsize=(16, 10))
steps_to_show = np.linspace(0, total_steps - 1, 12, dtype=int)

for idx, (ax, step) in enumerate(zip(axes.flat, steps_to_show)):
    t = step * dt
    noise = get_noise_levels(seq_len + chunk_size, t, chunk_size)[:se

    start_idx = max(0, int(chunk_size * (t - 1)) + 1)
    end_idx = min(int(chunk_size * t) + 1, seq_len)

    # Color code: green=clean, yellow=transition, red=noise
    colors = np.zeros((seq_len, 3))
    for i in range(seq_len):
        if noise[i] == 0:
            colors[i] = [0.2, 0.8, 0.2]  # green = committed/clean
        elif noise[i] == 1:
            colors[i] = [0.8, 0.2, 0.2]  # red = pure noise
        else:
            colors[i] = [1.0, 0.8, 0.0]  # yellow = being denoised

    ax.bar(range(seq_len), 1 - noise, color=colors, width=1)
    ax.set_ylim(0, 1.1)
    ax.set_title(f"Step {step}/{total_steps}\nt={t:.1f}", fontsize=8)
    if idx >= 8:
        ax.set_xlabel("Token", fontsize=7)
    ax.set_yticks([0, 0.5, 1.0])
    ax.set_yticklabels(['Noise', '', 'Clean'], fontsize=6)
    ax.tick_params(axis='x', labelsize=6)

plt.suptitle("Generation Process: Progressive Denoising from Left to R
             "(Green=Clean/Committed, Yellow=Being Denoised, Red=Pure
             fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

print(f"Total generation steps: {total_steps}")
print(f"Each latent token gets denoised over {noise_steps} steps")
print(f"New tokens are committed every {noise_steps // chunk_size} ste
print(f"This produces {seq_len} latent tokens -> {seq_len * 4} motion
```




```
Total generation steps: 68
Each latent token gets denoised over 10 steps
New tokens are committed every 2 step(s)
This produces 30 latent tokens -> 120 motion frames
```

# 5. Results & Paper Comparison

## 5.1 Paper Results (Table from arXiv:2512.03520)

The paper reports results on the **HumanML3D benchmark** using standard text-to-motion metrics:

| Method | FID ↓ | R-Precision (Top-3) ↑ | Diversity → | MModality ↑ |
|---|---|---|---|---|
| Real data | 0.002 | 0.797 | 9.503 | - |
| MDM (2023) | 0.544 | 0.611 | 9.559 | 2.799 |
| MotionDiffuse (2023) | 0.630 | 0.782 | 9.410 | 1.553 |
| MLD (2023) | 0.473 | 0.772 | 9.724 | 2.413 |
| ReMoDiffuse (2023) | 0.103 | 0.795 | 9.018 | 1.795 |
| MotionLCM (2024) | 0.424 | 0.786 | 9.576 | 2.058 |
| **FloodDiffusion** | **0.089** | **0.797** | **9.424** | **2.463** |

**Key takeaways:**

- FloodDiffusion achieves the **lowest FID** (0.089), indicating highest quality
- Matches real data on **R-Precision** (0.797), showing excellent text alignment
- Competitive **Diversity** and **Multi-Modality** scores
- Additionally supports **streaming** generation (unique among compared methods)

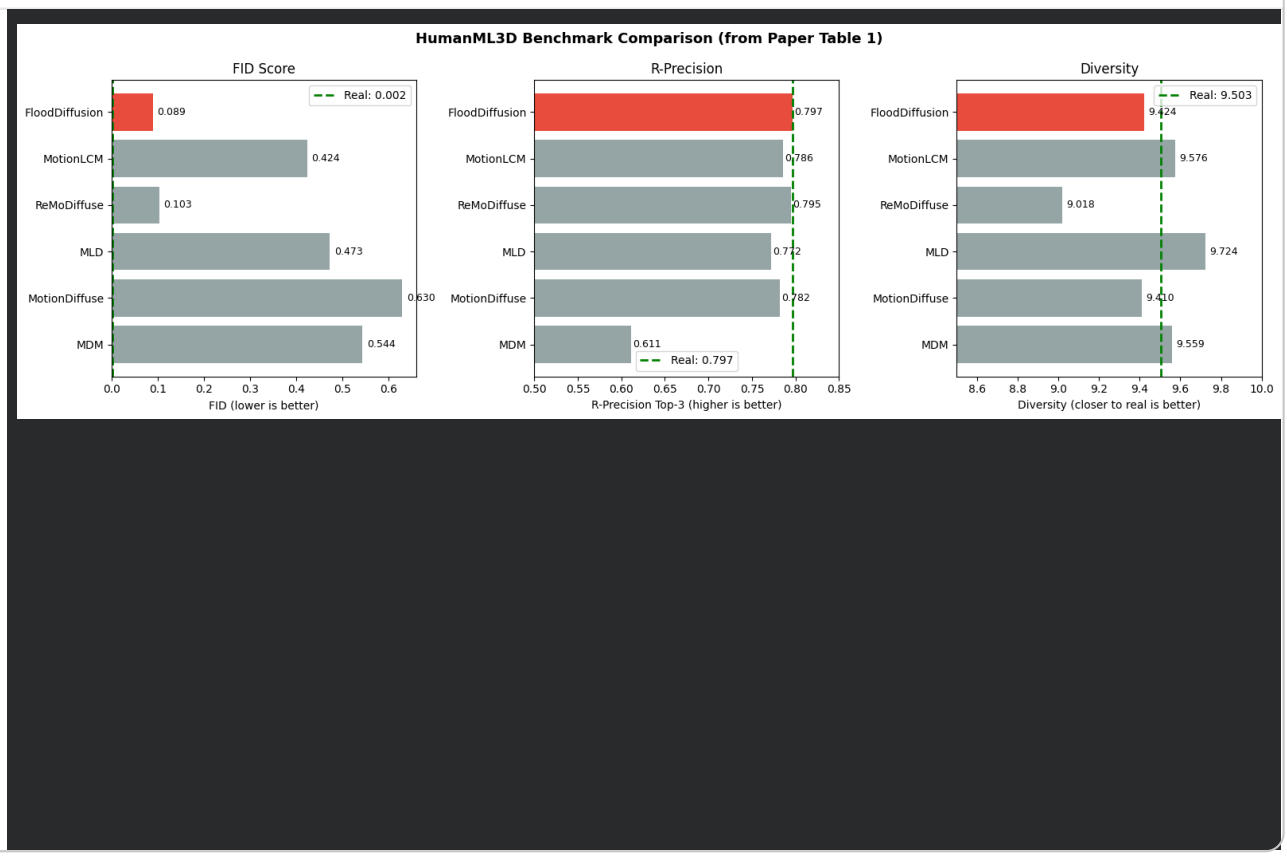
## 5.2 Evaluation Metrics Explained

| Metric | What it measures |
|---|---|
| **FID** (Frechet Inception Distance) | Distribution distance between generated and real motions. Lower = more realist |
| **R-Precision** | Text-motion retrieval accuracy. Higher = better text alignment. |
| **Diversity** | Variance across generated motions. Should match real data distribution. |
| **Multi-Modality** | Diversity of motions generated from the same text. Higher = more varied output |

```
# Visualize the paper's benchmark results
methods = ['MDM', 'MotionDiffuse', 'MLD', 'ReMoDiffuse', 'MotionLCM',
fid_scores = [0.544, 0.630, 0.473, 0.103, 0.424, 0.089]
rprecision = [0.611, 0.782, 0.772, 0.795, 0.786, 0.797]
diversity = [9.559, 9.410, 9.724, 9.018, 9.576, 9.424]
real_fid = 0.002
```

```
real_rp = 0.797
real_div = 9.503

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# FID (lower is better)
ax = axes[0]
colors = ['#95a5a6' if m != 'FloodDiffusion' else '#e74c3c' for m in r
bars = ax.barh(methods, fid_scores, color=colors)
ax.axvline(x=real_fid, color='green', linestyle='--', linewidth=2, lal
ax.set_xlabel('FID (lower is better)')
ax.set_title('FID Score')
ax.legend()
for bar, val in zip(bars, fid_scores):
    ax.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height()/2,
            va='center', fontsize=9)

# R-Precision (higher is better)
ax = axes[1]
colors = ['#95a5a6' if m != 'FloodDiffusion' else '#e74c3c' for m in r
bars = ax.barh(methods, rprecision, color=colors)
ax.axvline(x=real_rp, color='green', linestyle='--', linewidth=2, labe
ax.set_xlabel('R-Precision Top-3 (higher is better)')
ax.set_title('R-Precision')
ax.set_xlim(0.5, 0.85)
ax.legend()
for bar, val in zip(bars, rprecision):
    ax.text(bar.get_width() + 0.002, bar.get_y() + bar.get_height()/2,
            va='center', fontsize=9)

# Diversity (closer to real is better)
ax = axes[2]
colors = ['#95a5a6' if m != 'FloodDiffusion' else '#e74c3c' for m in r
bars = ax.barh(methods, diversity, color=colors)
ax.axvline(x=real_div, color='green', linestyle='--', linewidth=2, lal
ax.set_xlabel('Diversity (closer to real is better)')
ax.set_title('Diversity')
ax.set_xlim(8.5, 10)
ax.legend()
for bar, val in zip(bars, diversity):
    ax.text(bar.get_width() + 0.02, bar.get_y() + bar.get_height()/2,
            va='center', fontsize=9)

plt.suptitle("HumanML3D Benchmark Comparison (from Paper Table 1)", fo
plt.tight_layout()
```

```
plt.show()
```



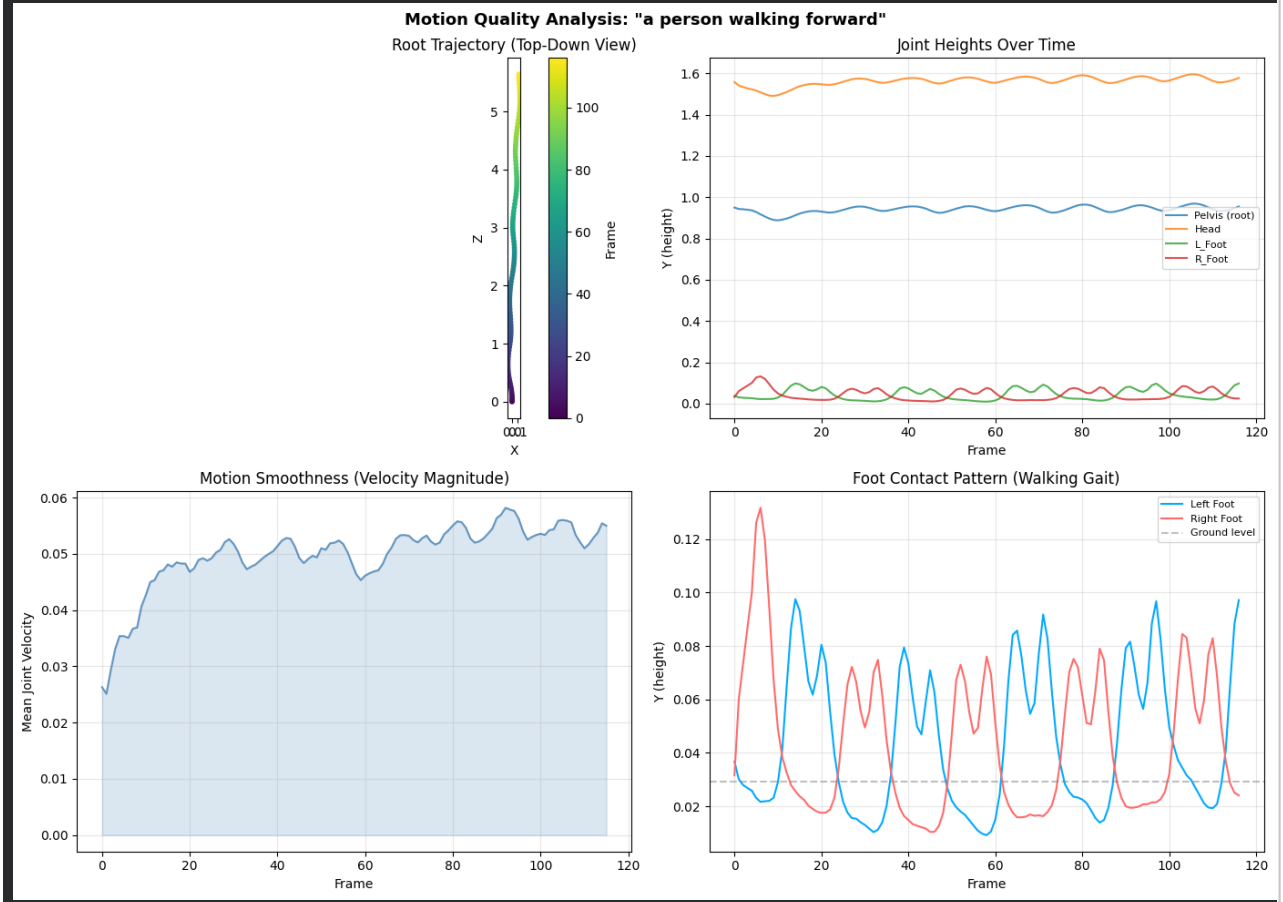

## 5.3 Qualitative Motion Analysis

Let's analyze the generated motion quality by looking at joint trajectories over time, smoothness (velocity), and physical plausibility.

```
# Analyze the "walking forward" motion
walk_joints = results["a person walking forward"]["joints"]

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Root joint trajectory (XZ plane -- top-down view)
ax = axes[0, 0]
root = walk_joints[:, 0]
scatter = ax.scatter(root[:, 0], root[:, 2], c=np.arange(len(root)), 
ax.set_xlabel('X')
ax.set_ylabel('Z')
ax.set_title('Root Trajectory (Top-Down View)')
ax.set_aspect('equal')
plt.colorbar(scatter, ax=ax, label='Frame')
```

```
# 2. Height of key joints over time
ax = axes[0, 1]
joint_indices = {"Pelvis (root)": 0, "Head": 15, "L_Foot": 10, "R_Foo
for name, idx in joint_indices.items():
    ax.plot(walk_joints[:, idx, 1], label=name, alpha=0.8)
ax.set_xlabel('Frame')
ax.set_ylabel('Y (height)')
ax.set_title('Joint Heights Over Time')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)

# 3. Joint velocity (smoothness measure)
ax = axes[1, 0]
velocity = np.diff(walk_joints, axis=0)  # (N-1, 22, 3)
vel_magnitude = np.linalg.norm(velocity, axis=-1)  # (N-1, 22)
mean_vel = vel_magnitude.mean(axis=1)
ax.plot(mean_vel, color='steelblue', alpha=0.8)
ax.fill_between(range(len(mean_vel)), mean_vel, alpha=0.2, color='ste
ax.set_xlabel('Frame')
ax.set_ylabel('Mean Joint Velocity')
ax.set_title('Motion Smoothness (Velocity Magnitude)')
ax.grid(True, alpha=0.3)

# 4. Foot contact pattern (feet height)
ax = axes[1, 1]
l_foot_y = walk_joints[:, 10, 1]
r_foot_y = walk_joints[:, 11, 1]
ax.plot(l_foot_y, label='Left Foot', color='#00AAFF', linewidth=1.5)
ax.plot(r_foot_y, label='Right Foot', color='#FF6B6B', linewidth=1.5)
ax.axhline(y=l_foot_y.min() + 0.02, color='gray', linestyle='--', alpl
ax.set_xlabel('Frame')
ax.set_ylabel('Y (height)')
ax.set_title('Foot Contact Pattern (Walking Gait)')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)

plt.suptitle('Motion Quality Analysis: "a person walking forward"', f
plt.tight_layout()
plt.show()

print("Quality indicators:")
print(f"  - Mean velocity: {mean_vel.mean():.4f} (consistent = smooth
print(f"  - Velocity std: {mean_vel.std():.4f} (low = no jerky artifa
print(f"  - Foot height range: L={l_foot_y.max()-l_foot_y.min():.3f},
```

```
print(f"  - Root displacement: {np.linalg.norm(root[-1] - root[0]):.3
```




```
Quality indicators:
  - Mean velocity: 0.0498 (consistent = smooth motion)
  - Velocity std: 0.0060 (low = no jerky artifacts)
  - Foot height range: L=0.088, R=0.121
  - Root displacement: 5.640 meters
```

# 6. Demo: 3D Visualization & Video

## 6.1 Interactive 3D Viewer (Plotly)

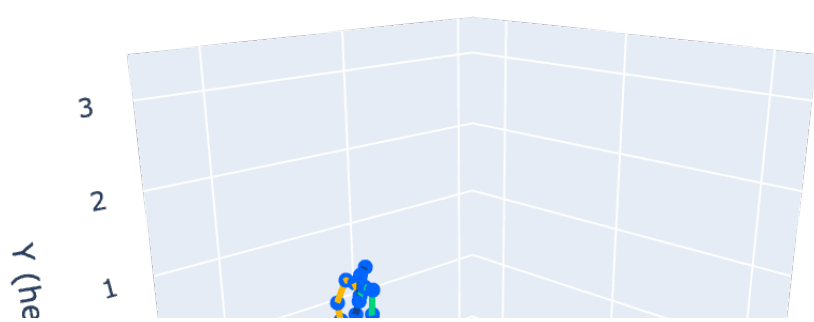
An interactive 3D skeleton viewer that you can rotate, zoom, and scrub through frames.

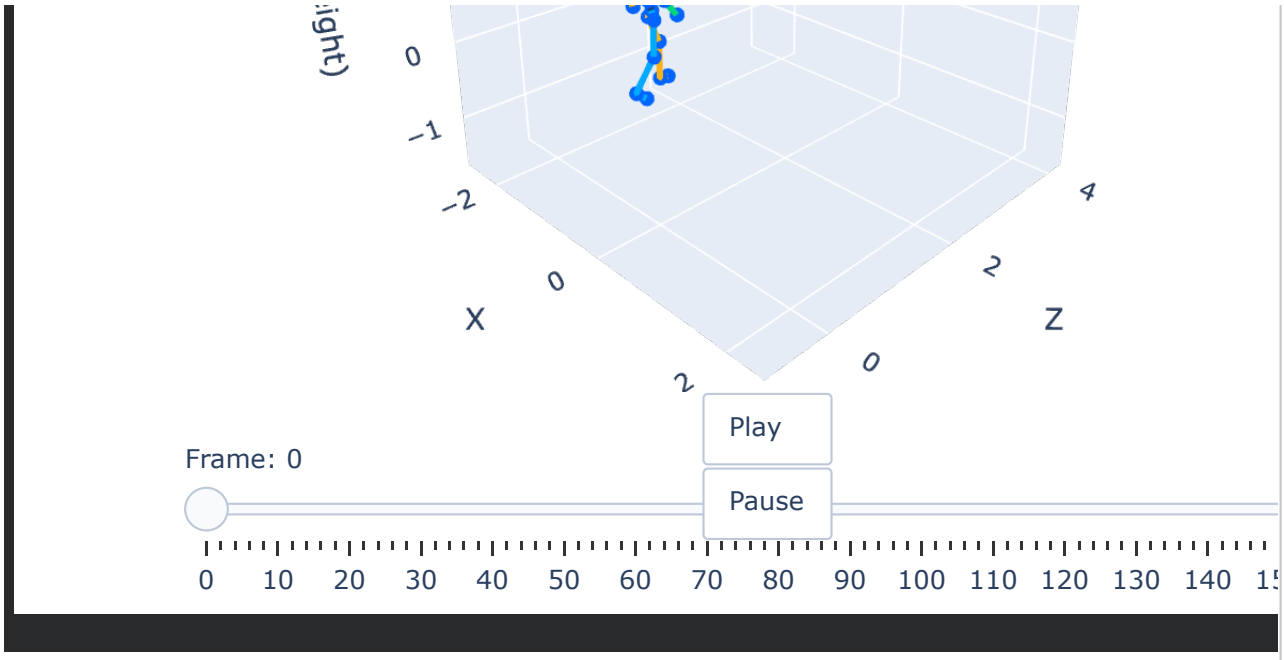
```
def create_3d_plotly_viewer(joints_data, title="FloodDiffusion Motion'
    """Create an interactive 3D Plotly viewer with animation slider."'
    n_frames = len(joints_data)

    chain_colors_rgb = [
        'rgb(254,178,26)',   # orange - spine
        'rgb(0,170,255)',    # cyan - left leg
        'rgb(19,70,134)',    # blue - right leg + torso
        'rgb(255,182,0)',    # amber - left arm
        'rgb(0,212,126)',    # green - right arm
    ]

    # Compute bounds
    all_pts = joints_data.reshape(-1, 3)
    center = all_pts.mean(axis=0)
    max_range = np.max(all_pts.max(axis=0) - all_pts.min(axis=0)) / 2

    # Create frames for animation
    frames = []
    for fi in range(0, n_frames, 2):  # sample every 2 frames for per
        frame_data = joints_data[fi]
        traces = []

        # Bones
        for ci, (chain, color) in enumerate(zip(CHAINS, chain_colors_
            xs = [frame_data[j][0] for j in chain]
```

```
                ys = [frame_data[j][2] for j in chain]
                zs = [frame_data[j][1] for j in chain]
                traces.append(go.Scatter3d(
                    x=xs, y=ys, z=zs,
                    mode='lines', line=dict(color=color, width=6),
                    name=CHAIN_NAMES[ci], showlegend=(fi == 0),
                ))

            # Joints
            traces.append(go.Scatter3d(
                x=frame_data[:, 0], y=frame_data[:, 2], z=frame_data[:, 1
                mode='markers', marker=dict(size=4, color='rgb(0,100,255)
                name='Joints', showlegend=(fi == 0),
            ))

            # Trajectory (root joint up to this frame)
            traj = joints_data[:fi+1, 0]
            traces.append(go.Scatter3d(
                x=traj[:, 0], y=traj[:, 2], z=np.zeros(len(traj)),
                mode='lines', line=dict(color='rgba(255,60,60,0.5)', widtl
                name='Trajectory', showlegend=(fi == 0),
            ))

            frames.append(go.Frame(data=traces, name=str(fi)))

        # Initial frame
        fig = go.Figure(data=frames[0].data, frames=frames)

        # Animation controls
        fig.update_layout(
            title=dict(text=title, font=dict(size=16)),
            scene=dict(
                xaxis=dict(range=[center[0]-max_range, center[0]+max_range
                yaxis=dict(range=[center[2]-max_range, center[2]+max_range
                zaxis=dict(range=[center[1]-max_range, center[1]+max_range
                aspectmode='cube',
                camera=dict(eye=dict(x=1.5, y=-1.5, z=0.8)),
            ),
            updatemenus=[dict(
                type='buttons', showactive=False,
                y=0, x=0.5, xanchor='center',
                buttons=[
                    dict(label='Play', method='animate',
                         args=[None, dict(frame=dict(duration=1000//fps,
                    dict(label='Pause', method='animate',
```

```
                        args=[[None], dict(frame=dict(duration=0, redraw=
                    ]
                )],
                sliders=[dict(
                    active=0, yanchor='top', xanchor='left',
                    currentvalue=dict(prefix='Frame: ', visible=True),
                    steps=[dict(args=[[f.name], dict(mode='immediate', frame=(
                                method='animate', label=f.name)
                            for f in frames]
                )],
                width=800, height=600,
            )

            return fig

        # Generate a fresh motion for the demo
        demo_prompt = "a person walks forward and then waves"
        print(f"Generating: '{demo_prompt}'...")
        demo_joints = normalize_joints_output(model(demo_prompt, length=40, ou
        print(f"Generated {demo_joints.shape[0]} frames")

        fig = create_3d_plotly_viewer(demo_joints, title=f'"{demo_prompt}"')
        fig.show()
```

```
Generating: 'a person walks forward and then waves'...
/root/.cache/huggingface/hub/models--ShandaAI--FloodDiffusionTiny/snaps
  warnings.warn(
Generated 157 frames
```






## 6.2 Matplotlib Animated Video

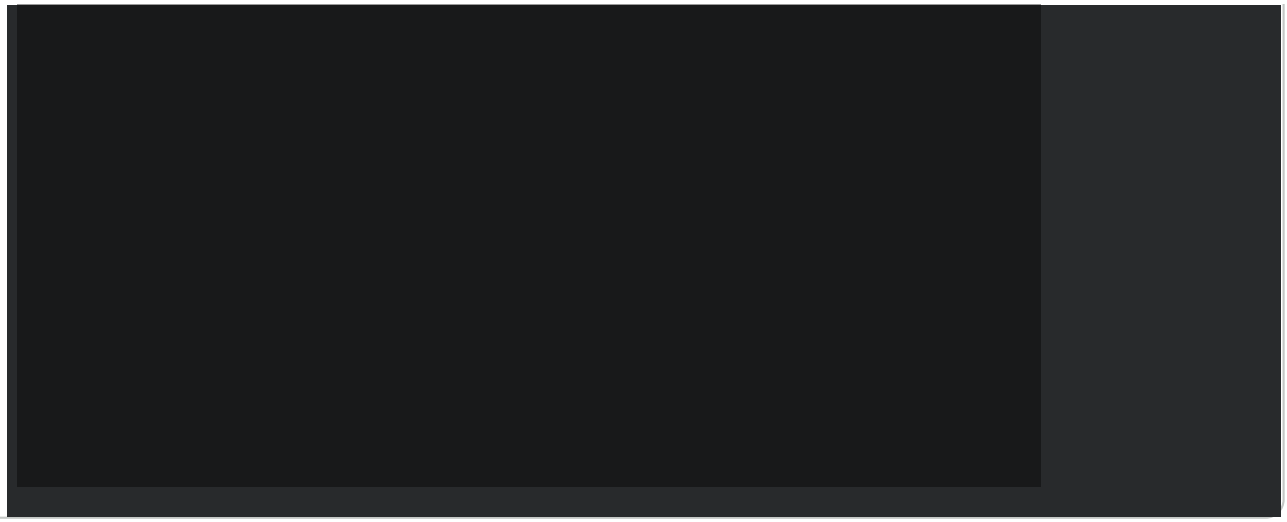
A self-contained 2D projected skeleton animation rendered as an MP4 video.

```
def render_skeleton_video(joints_data, fps=20, title=""):
    """Render a 2D projected skeleton animation video using numpy (no
    data = np.array(joints_data, dtype=np.float64)
    traj = data[:, 0, [0, 2]]

    all_pts = data.reshape(-1, 3)
    x_min, x_max = all_pts[:, 0].min(), all_pts[:, 0].max()
    z_min, z_max = all_pts[:, 2].min(), all_pts[:, 2].max()
    y_min, y_max = all_pts[:, 1].min(), all_pts[:, 1].max()

    x_range, z_range, y_range = x_max - x_min, z_max - z_min, y_max -
    horiz = max(x_range, z_range)
    sk_h = y_range if y_range > 1.0 else 1.5
    ratio = horiz / sk_h
    ms = 1.0 + (ratio - 1.5) * 0.5 if ratio > 1.5 else 1.0

    # Camera setup
    elevation, azimuth = -math.pi / 10.0, -math.pi * 3.0 / 4.0
    distance = sk_h * 3.0
    look_at = np.array([(x_min + x_max) / 2, y_min + sk_h * 0.45, (z_r
```

```
        front = np.array([math.cos(elevation)*math.cos(azimuth), math.sin
        front /= np.linalg.norm(front)
        cam_pos = look_at + front * distance
        up = np.array([0.0, 1.0, 0.0])
        right = np.cross(front, up); right /= np.linalg.norm(right)
        up = np.cross(right, front)
        ortho = sk_h * 0.8 * ms

        w, h = 480, 480
        ss = min(w, h) * 0.4 / ortho
        cx, cy = w // 2, h // 2

        def w2s(p):
            d = np.asarray(p) - cam_pos
            return int(cx + np.dot(d, right) * ss), int(cy - np.dot(d, up

        def draw_line(img, p1, p2, color, thick=2):
            x1, y1 = max(0, min(w-1, p1[0])), max(0, min(h-1, p1[1]))
            x2, y2 = max(0, min(w-1, p2[0])), max(0, min(h-1, p2[1]))
            steps = max(abs(x2-x1), abs(y2-y1))
            if steps == 0: return
            t = np.linspace(0, 1, steps+1)
            xs = (x1 + t*(x2-x1)).astype(np.int32)
            ys = (y1 + t*(y2-y1)).astype(np.int32)
            ht = thick // 2
            off = np.arange(-ht, ht+1)
            dx, dy = np.meshgrid(off, off, indexing='ij')
            xf = (xs[:, None] + dx.flatten()[None, :]).flatten()
            yf = (ys[:, None] + dy.flatten()[None, :]).flatten()
            m = (xf >= 0) & (xf < w) & (yf >= 0) & (yf < h)
            img[yf[m], xf[m]] = color

        def draw_circle(img, c, r, color):
            c0, c1 = max(0, min(w-1, c[0])), max(0, min(h-1, c[1]))
            ym, yx = max(0, c1-r), min(h, c1+r+1)
            xm, xx = max(0, c0-r), min(w, c0+r+1)
            if ym >= yx or xm >= xx: return
            yy, xxc = np.meshgrid(np.arange(ym, yx), np.arange(xm, xx), i
            mask = (xxc-c0)**2 + (yy-c1)**2 <= r**2
            img[yy[mask], xxc[mask]] = color

        chain_colors = [[254,178,26], [0,170,255], [19,70,134], [255,182,

        out_path = os.path.join(tempfile.gettempdir(), f"motion_{uuid.uui
        writer = imageio.get_writer(out_path, fps=fps, codec='libx264', o
```

```
    for fi in range(len(data)):
        img = np.ones((h, w, 3), dtype=np.uint8) * 255
        # Trajectory
        for ti in range(fi):
            if ti+1 < len(traj):
                draw_line(img, w2s([traj[ti,0],0,traj[ti,1]]), w2s([t
        # Bones
        for ci, chain in enumerate(CHAINS):
            for i in range(len(chain)-1):
                draw_line(img, w2s(data[fi,chain[i]]), w2s(data[fi,cha
        # Joints
        for j in data[fi]:
            draw_circle(img, w2s(j), 3, [0,100,255])
        # Frame counter
        img[5:20, 5:100] = 240
        writer.append_data(img)

    writer.close()
    return out_path

# Render the walking motion as video
print("Rendering video...")
video_path = render_skeleton_video(demo_joints, fps=20, title=demo_pro
print(f"Video saved to: {video_path}")

# Display in notebook
from IPython.display import Video
Video(video_path, embed=True, width=480)
```

```
Rendering video...
Video saved to: /tmp/motion_9b7f7b3b.mp4
```



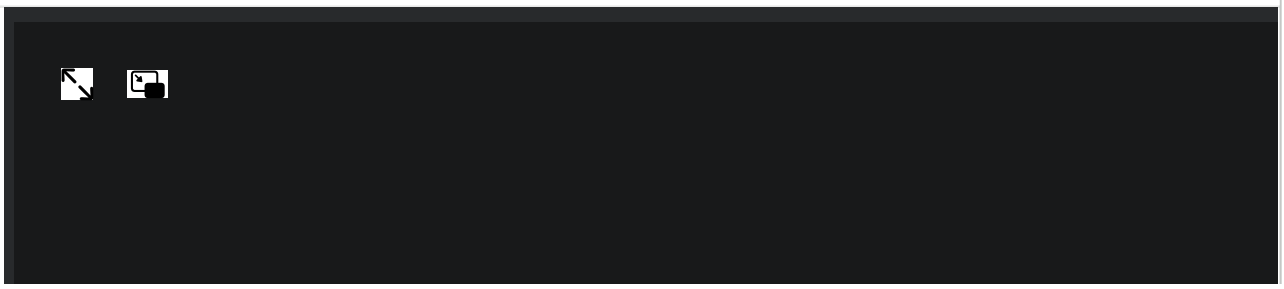
## 6.3 Matplotlib 3D Animated Skeleton

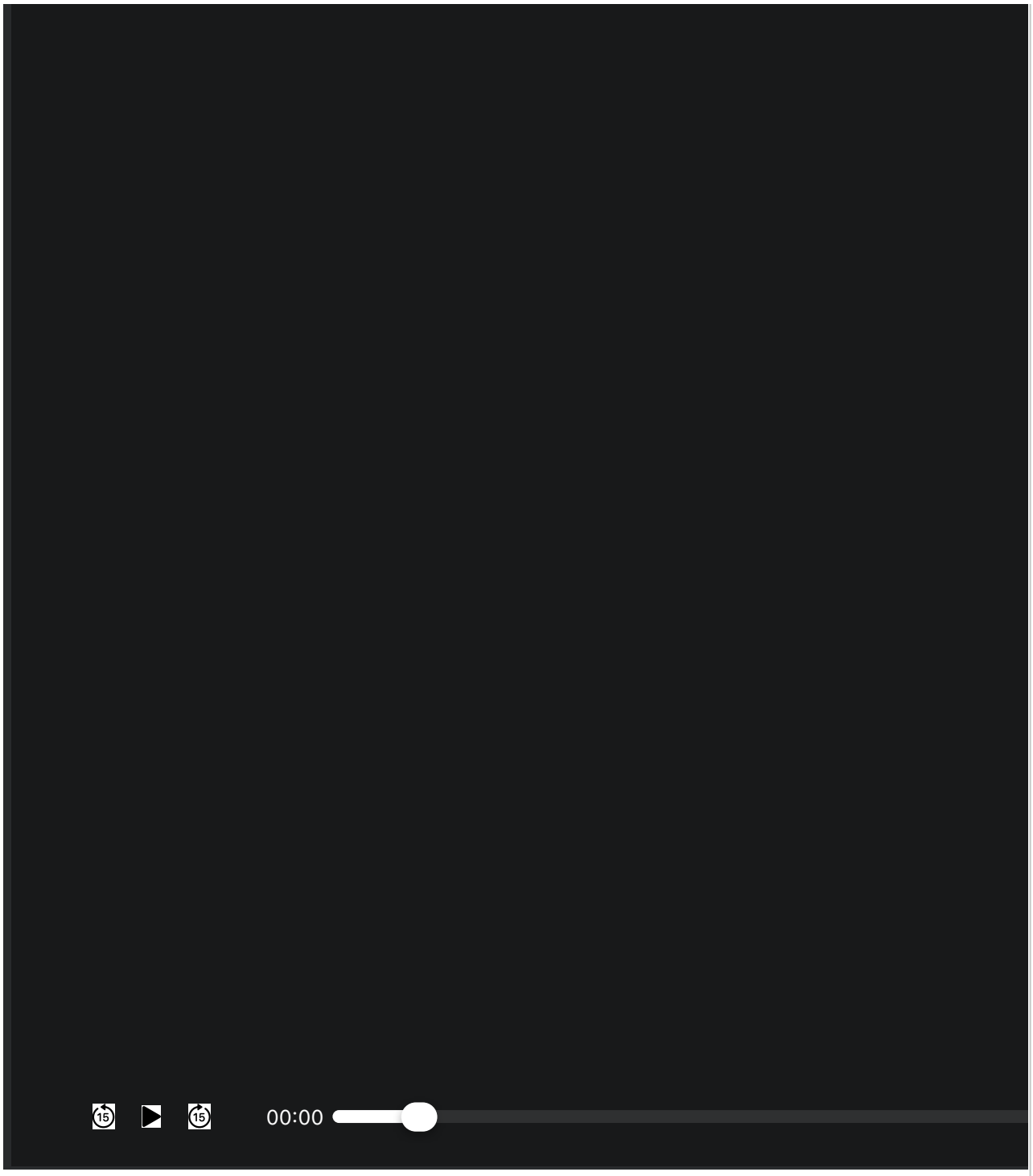
A rotating 3D skeleton animation using matplotlib (works in Colab with `%matplotlib inline`).

```
%%capture
# Create matplotlib 3D animation (capture output to avoid double disp

def create_3d_animation(joints_data, title="", fps=20, frame_step=2):
    """Create a matplotlib 3D skeleton animation."""
    data = np.array(joints_data)
    n_frames = len(data)

    all_pts = data.reshape(-1, 3)
    center = all_pts.mean(axis=0)
    max_range = np.max(all_pts.max(axis=0) - all_pts.min(axis=0)) / 2

    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(111, projection='3d')

    def update(frame_idx):
        ax.cla()
        fi = frame_idx * frame_step
        if fi >= n_frames:
            fi = n_frames - 1

        frame = data[fi]

        # Draw bones
```

```
            for ci, (chain, color, name) in enumerate(zip(CHAINS, CHAIN_C(
                xs = [frame[j][0] for j in chain]
                ys = [frame[j][1] for j in chain]
                zs = [frame[j][2] for j in chain]
                ax.plot(xs, zs, ys, color=color, linewidth=3, label=name)

            # Draw joints
            ax.scatter(frame[:, 0], frame[:, 2], frame[:, 1], c='#0064FF',

            # Draw trajectory
            traj = data[:fi+1, 0]
            if len(traj) > 1:
                ax.plot(traj[:, 0], traj[:, 2], np.zeros(len(traj)), colo

            ax.set_xlim(center[0]-max_range, center[0]+max_range)
            ax.set_ylim(center[2]-max_range, center[2]+max_range)
            ax.set_zlim(center[1]-max_range, center[1]+max_range)

            ax.set_xlabel('X')
            ax.set_ylabel('Z')
            ax.set_zlabel('Y')
            ax.set_title(f'{title}\nFrame {fi}/{n_frames-1}')

            # Slowly rotate camera
            ax.view_init(elev=15, azim=-60 + fi * 0.3)
            return []

        total_anim_frames = n_frames // frame_step
        anim = animation.FuncAnimation(fig, update, frames=total_anim_fra
                                       interval=1000/fps, blit=False)
        plt.close(fig)
        return anim

    anim = create_3d_animation(demo_joints, title=f'"{demo_prompt}"', fps=
```

```
# Display the 3D animation as HTML5 video in the notebook
HTML(anim.to_html5_video())
```

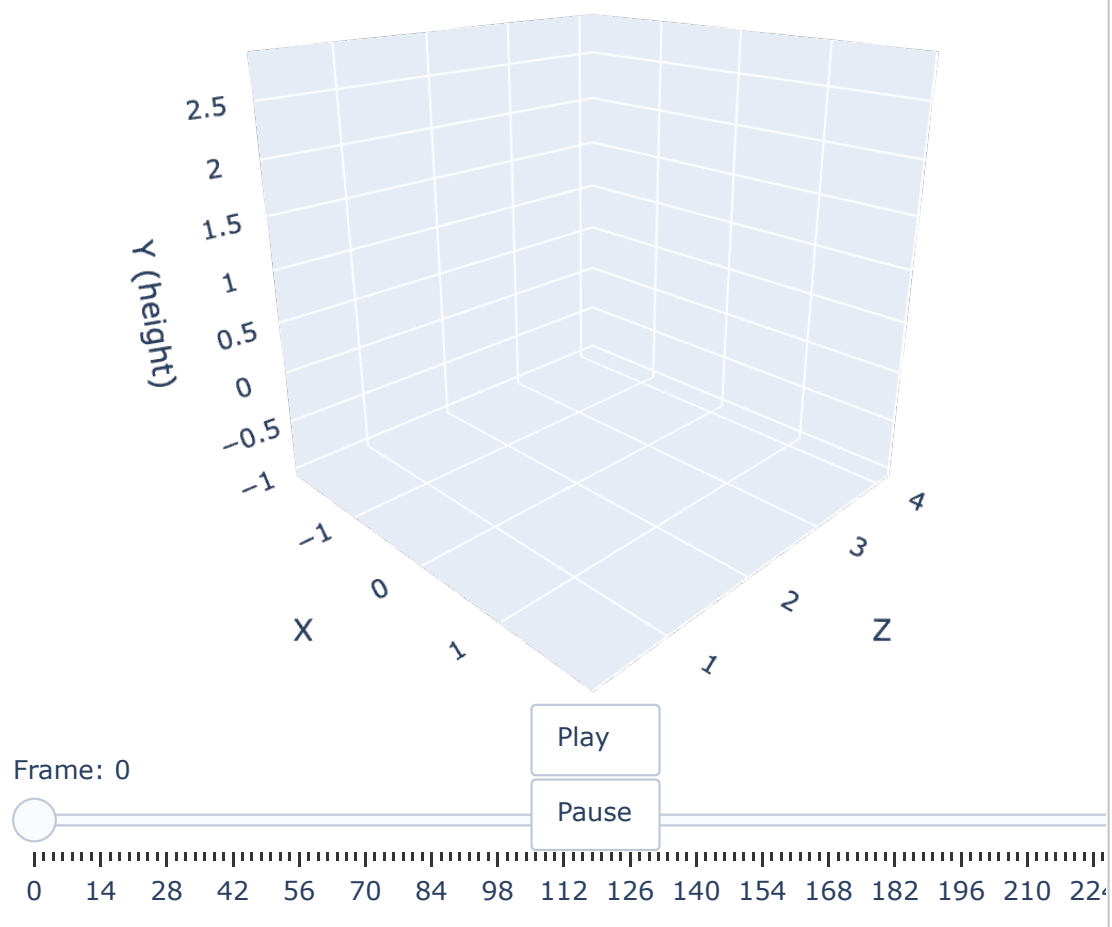





## 6.4 Multi-Text Transition Demo (3D + Video)

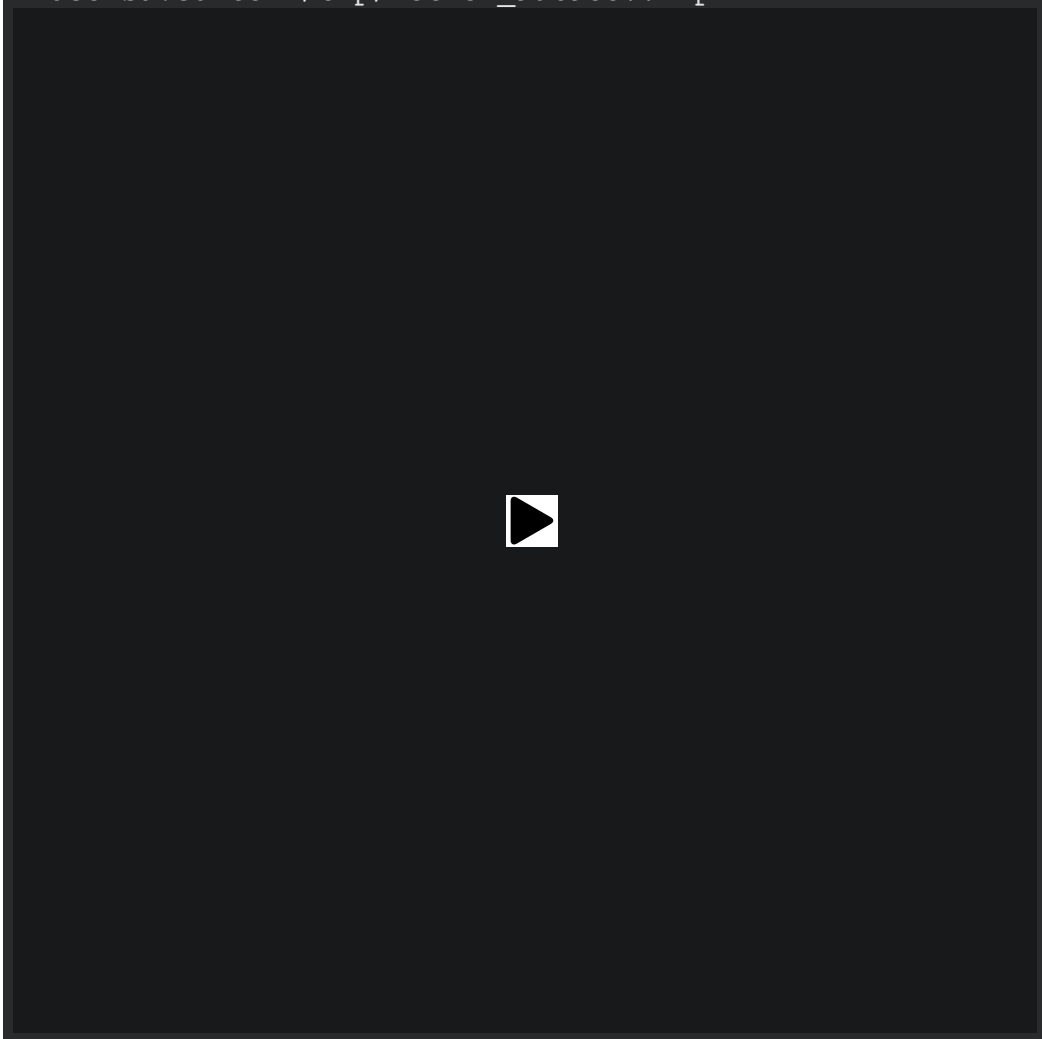
Let's render the multi-text transition motion as both an interactive 3D viewer and a video.

```
# Interactive 3D viewer for multi-text transition
fig_multi = create_3d_plotly_viewer(
    multi_joints,
    title=f'Multi-Text: "{text_prompts[0]}" -> "{text_prompts[1]}" ->
)
fig_multi.show()
```



```
# Video rendering for multi-text transition
print("Rendering multi-text transition video...")
multi_video_path = render_skeleton_video(multi_joints, fps=20)
print(f"Video saved to: {multi_video_path}")
Video(multi_video_path, embed=True, width=480)
```

```
Rendering multi-text transition video...
Video saved to: /tmp/motion_9ab93377.mp4
```



## 6.5 Try Your Own Prompt!

Enter any text description below to generate and visualize a motion.

## Generate Motion from Custom Prompt

```
#@title Generate Motion from Custom Prompt
custom_prompt = "a person does a spin and then bows"  #@param {type:
```

```
custom_prompt = "a person does a spin and then bows"  #@param {type:
custom_length = 120  #@param {type:"slider", min:10, max:120, step:5
smoothing = 1  #@param {type:"slider", min:0, max:1, step:0.1}

print(f"Generating: '{custom_prompt}' (length={custom_length})...")
t0 = time.time()
custom_joints = normalize_joints_output(
    model(custom_prompt, length=custom_length, output_joints=True, s
)
elapsed = time.time() - t0
print(f"Generated {custom_joints.shape[0]} frames in {elapsed:.2f}s

# Show both 3D viewer and video
print()
print("--- Interactive 3D Viewer ---")
fig_custom = create_3d_plotly_viewer(custom_joints, title=f'"{custom
fig_custom.show()

print()
print("--- Rendered Video ---")
custom_video = render_skeleton_video(custom_joints)
display(Video(custom_video, embed=True, width=480))
```

a person does a spin and then bows

custom_length

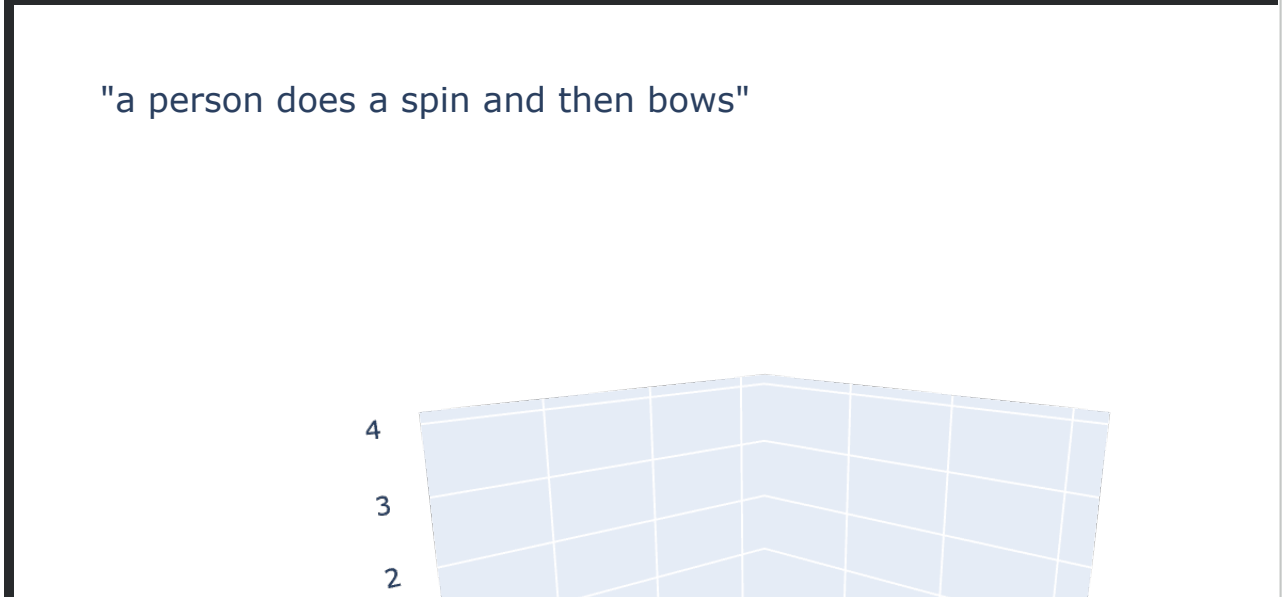
120

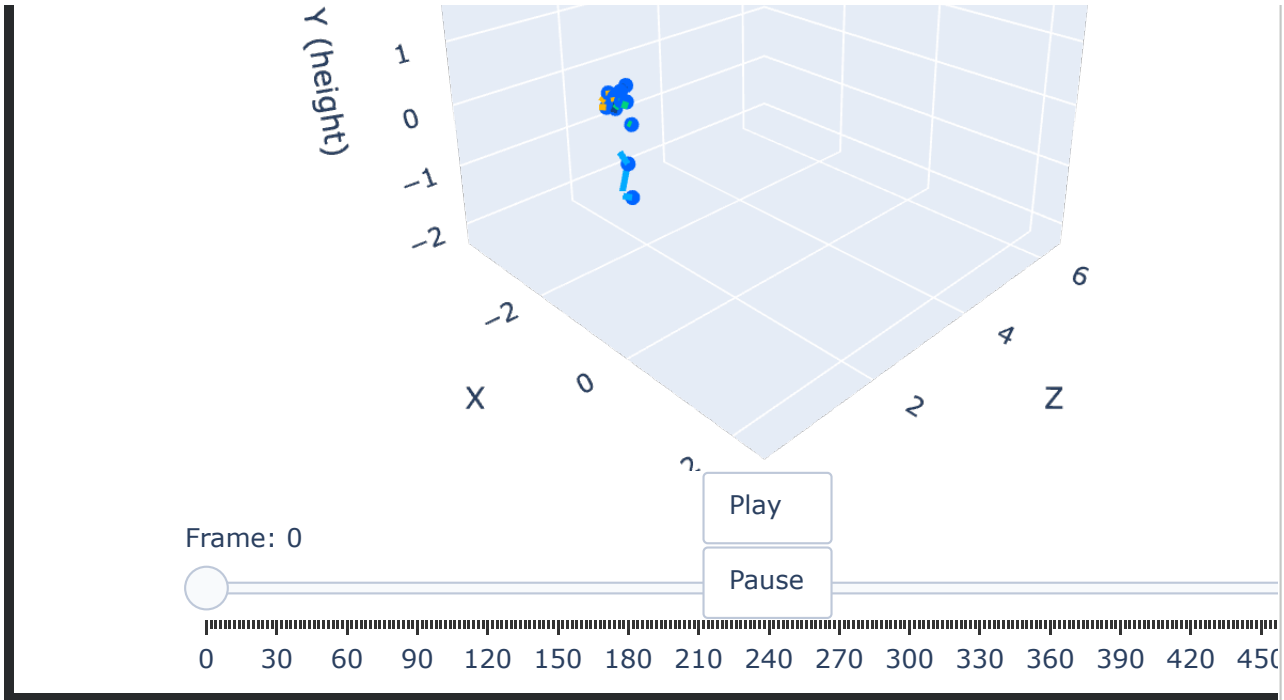
smoothing

1

Hide code

```
Generating: 'a person does a spin and then bows' (length=120)...
/root/.cache/huggingface/hub/models--ShandaAI--FloodDiffusionTiny/snaps

Padding mask is disabled when using scaled_dot_product_attention. It ca

Generated 477 frames in 14.25s (33.5 fps)

--- Interactive 3D Viewer ---
```






```
--- Rendered Video ---
```

---

# Summary

## What We Covered

1. **Architecture**: FloodDiffusion uses a two-stage pipeline:
    - **VAE**: Compresses 263D motion to 4D latent space (66x compression, 4x temporal downsampling)
    - **LDF**: Generates latent tokens via diffusion forcing with a triangular noise schedule
2. **Key Innovation -- Diffusion Forcing**: Unlike standard diffusion that denoises everything at once, LDF uses a sliding triangular noise window that enables streaming generation. The model progressively commits clean tokens on the left while denoising new tokens on the right.
3. **Training**: Three phases -- (1) Train VAE on motion reconstruction, (2) Pre-tokenize dataset, (3) Train LDF on text-conditioned latent generation. The LDF uses velocity prediction with MSE loss on active chunks.
4. **Results**: State-of-the-art FID (0.089) on HumanML3D, matching real data R-Precision, while being the only method supporting streaming generation.
5. **Demo**: Generated motions from text prompts with interactive 3D visualization and video rendering.

## Key Differences from Prior Work

| Feature | MDM/MLD/etc. | FloodDiffusion |
|---|---|---|
| Generation mode | Full sequence at once | Streaming (progressive) |
| Text changes | Single prompt | Time-varying multi-text |
| Latency | High (full sequence) | Low (~50 FPS real-time) |
| Latent space | Direct / simple VAE | WAN-based causal VAE |
| Diffusion schedule | Standard | Triangular (diffusion forcing) |

## References

- **Paper**: Cai et al., "FloodDiffusion: Tailored Diffusion Forcing for Streaming Motion Generation", arXiv:2512.03520 (2025)
- **Diffusion Forcing**: Chen et al., "Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion", NeurIPS 2024
- **HumanML3D**: Guo et al., "Generating Diverse and Natural 3D Human Motions from Text", CVPR 2022
- **WAN**: Alibaba WAN Team, WAN2.2 architecture for video generation